Security-Aware RowHammer Mitigation in a DRAM Model

Zefu Li
School of Engineering and Applied Science
University of Pennsylvania

***Abstract*—RowHammer is a DRAM disturbance effect in which repeatedly activating a small set of rows can induce bit flips in adjacent victim rows. In this project, we implement a RowHammer fault model and several mitigation policies as a controller plugin on top of a DDR4-based simulation framework. The logical model captures activation-based hammer counting, victim-row disturbance accumulation, optional row-space compression, and deterministic or probabilistic bit-flip injection. We then extend this framework with a timing-aware maintenance surrogate: once a protection action is selected by the plugin, it is injected into the controller priority path as a row-level open/close maintenance sequence, allowing the simulator to expose performance-related overhead such as average read latency, memory-system cycles, and priority-queue occupancy. On top of this model, we study four mitigation policies: uniform Target Row Refresh (TRR), security-aware TRR, uniform Probabilistic Adjacent Row Activation (PARA), and security-aware PARA. Using a focused timing-aware parameter sweep, we show that security-aware TRR benefits from moderate critical-row thresholds combined with relaxed non-critical thresholds, while security-aware PARA benefits from maintaining low non-critical protection probabilities and substantially higher critical-row probabilities. These results reveal not only which policies reduce critical-region bit flips, but also how the resulting protection strength trades off against timing overhead.**

***Index Terms*—RowHammer, DRAM, TRR, PARA**

I. Introduction

Modern DRAM devices are increasingly susceptible to disturbance effects as cell sizes shrink and arrays become more densely packed. A prominent example is the RowHammer phenomenon, in which repeatedly activating a small set of aggressor rows can induce bit flips in adjacent victim rows, as illustrated in Figure 1. Such disturbance-induced bit flips can undermine both memory reliability and memory-system security.

Several mitigation techniques have been proposed to reduce RowHammer risk. A straightforward approach is to increase the global refresh rate so that disturbed cells are restored more frequently, at the cost of higher energy consumption and reduced effective memory bandwidth. Target Row Refresh (TRR) is more selective: once a row is activated unusually often, the controller issues targeted maintenance to that row or its neighbors. Another lightweight direction is Probabilistic Adjacent Row Activation (PARA), in which adjacent rows are protected with a small probability when an activation event occurs. TRR is stateful and threshold-driven, while PARA is comparatively stateless and probabilistic. Both mechanisms trade off protection strength against overhead and implementation complexity.

Most existing schemes, however, treat DRAM rows uniformly. They do not distinguish between rows whose corruption mainly affects correctness and rows whose corruption directly compromises security. In practice, corruption of page-table-like metadata or other privileged state is often far more damaging than corruption of ordinary user data. We study this question through a simulation-based framework and compare uniform and security-aware versions of TRR and PARA under the same fault model.

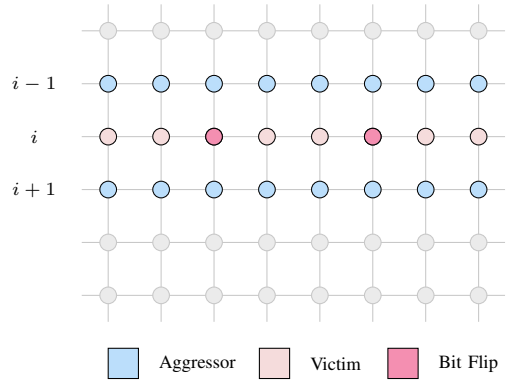

Fig. 1. Illustration of RowHammer in a DRAM array. Frequent activations of rows $i-1$ and $i+1$ induce bit flips in victim row i.

II. Threat Model

We consider an untrusted software adversary that can generate memory-access patterns of its choice. The adversary cannot physically tamper with the DRAM chips, but can produce RowHammer-style access patterns that repeatedly activate selected aggressor rows. Its goal is to induce bit flips in victim rows corresponding to security-critical regions rather than ordinary user data.

Under this model, our objective is to design and evaluate mitigation policies that minimize RowHammer-induced bit flips in security-critical rows while keeping protection overhead manageable.

III. System Model

We use a DRAM model built on top of a DDR4 timing configuration in a cycle-level simulator. The underlying simulator is Ramulator 2.0 [2], a modern, modular, extensible, and cycle-accurate DRAM simulator designed to support rapid implementation and evaluation of memory-controller and DRAM design changes. Rather than attempting to reproduce all circuit-level details of disturbance physics, our added model tracks logical rows, activation events, adjacent-row disturbance accumulation, and policy-triggered protection actions.

A. Logical Fault Model

The key fault-model assumption is that RowHammer is driven by repeated row activations rather than ordinary accesses to an already-open row. For each logical row, the controller plugin maintains an activation counter. When the activation count of a row reaches a configurable hammer threshold, we treat this as one effective hammer event. Each hammer event increases the disturbance level of the two adjacent rows by one unit. Thus, for aggressor row r, a hammer event increments the disturbance state of victim rows $r-1$ and $r+1$.

Once disturbance accumulates on a victim row, the model determines whether a bit flip occurs. We support both deterministic and probabilistic modes. In deterministic mode, a bit flip occurs once the disturbance level reaches a fixed threshold. In probabilistic mode, once the disturbance level reaches a configurable onset point, the bit-flip probability is computed using one of three functions: step, linear, or sigmoid.

To make policy differences visible under sparse traces, we optionally compress raw row identifiers into a smaller logical row space. In the current implementation, this compression remains part of the logical RowHammer model: multiple raw rows may map to the same logical row, increasing interaction density and making disturbance accumulation more likely. This is an experimental stress knob rather than a native DRAM remapping mechanism.

B. Timing-Aware Maintenance Surrogate

A limitation of the original logical plugin is that protection actions are recorded only as logical events, which is sufficient to study effectiveness but does not expose timing overhead. To address this, we introduce a first-order timing-aware maintenance surrogate.

When the plugin decides that a protection action should occur, it no longer clears the victim disturbance state immediately. Instead, it injects a high-priority row-level maintenance sequence into the controller priority path. In the current implementation, each protection action is approximated by a row-level open-row / close-row sequence targeting the victim row. This sequence occupies the controller priority queue and is issued through the native command scheduling path, allowing the simulator to expose timing-related effects such as increased average read latency, increased memory-system cycles, and non-zero priority-queue occupancy.

This surrogate is intentionally lightweight. It is more realistic than a purely logical counter model, but it should still be interpreted as a first-order timing-aware approximation rather than a fully faithful targeted-refresh implementation.

C. Critical and Non-Critical Regions

We partition logical rows into security-critical and non-critical classes. A simple periodic labeling rule is used: rows satisfying a stride-and-offset condition are marked as critical, while all others are treated as non-critical. This synthetic partition is sufficient to evaluate whether security-aware policies actually bias protection toward critical rows and whether they reduce critical-region bit flips.

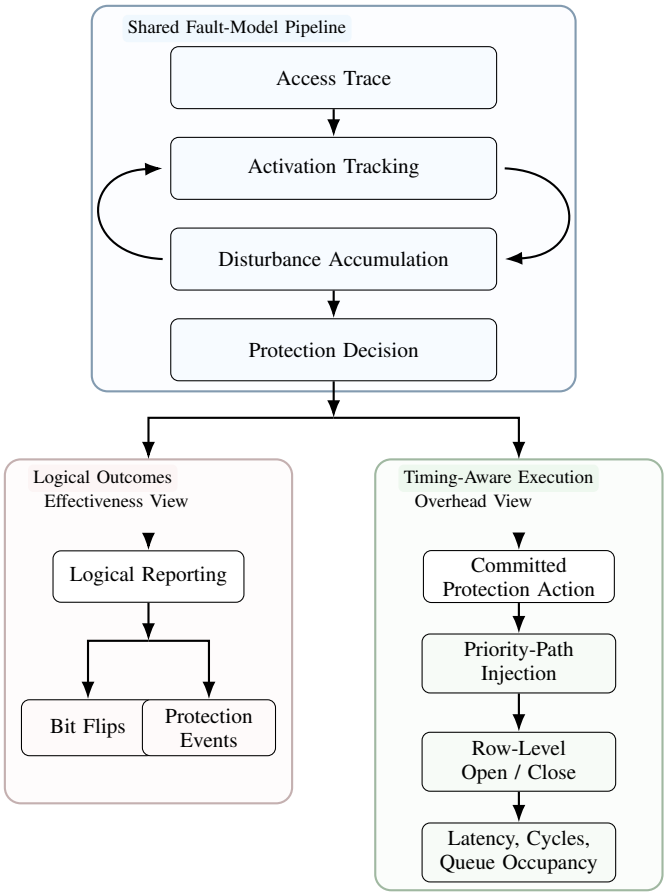

Fig. 2. Timing-aware integration flow. A shared fault-model pipeline processes each access through activation tracking, recursive disturbance-state updating, and protection decision. The recursive interaction between activation tracking and disturbance accumulation is illustrated by the bidirectional feedback notation. The decision then feeds two complementary views: a logical-outcome branch that reports bit flips and protection events, and a timing-aware execution branch that commits protection actions through priority-path injection and a row-level open/close surrogate, exposing latency, cycle, and queue-occupancy overheads.

IV. Mitigation Policies

A. Uniform Target Row Refresh

Uniform TRR is a stateful threshold-based policy. Each victim row is assigned the same disturbance threshold. When the disturbance of a victim row reaches this threshold, the controller schedules a protection action for that row using the same rule regardless of whether the row is critical or non-critical.

B. Security-Aware Target Row Refresh

Security-aware TRR uses different thresholds for critical and non-critical rows. Critical rows receive stronger protection by triggering maintenance at a lower disturbance level. Non-critical rows are protected less aggressively. This biases row-level maintenance toward security-critical rows.

C. Uniform Probabilistic Adjacent Row Activation

Uniform PARA is a stateless probabilistic policy. Following the PARA concept introduced by Kim *et al.* [1], a row opening/closing event can probabilistically trigger protection of an adjacent row without maintaining explicit per-row activation counters for the policy itself. In our model, whenever a hammer event affects a victim row, that victim is protected with a fixed probability. The same probability rule is applied to both critical and non-critical rows.

D. Security-Aware Probabilistic Adjacent Row Activation

Security-aware PARA extends the same stateless mechanism but assigns different probabilities to critical and non-critical victims. Critical rows are protected with a higher probability than non-critical rows, biasing maintenance toward critical regions.

V. Evaluation Methodology

A. Metrics

For each configuration, we record total bit flips, bit flips in critical rows, bit flips in non-critical rows, and the number of committed protection actions. For TRR, we separately count actions applied to critical and non-critical rows. For PARA, we separately count successful probabilistic protection events applied to critical and non-critical rows.

To expose timing overhead, we additionally collect average read latency, total memory-system cycles, and average priority-queue occupancy from the simulator. These metrics are not full area/power measurements; rather, they serve as performance-related overhead proxies for the row-level maintenance surrogate introduced in this work.

B. Fault-Model Sweep for Parameter Selection

To determine which fault settings are most informative before the detailed timing-aware policy sweep, we first perform a focused fault-model sweep under representative policy operating points. In this sweep, the probability function is fixed to sigmoid, the hammer threshold is varied over $\{1, 2, 3, 4\}$, the probabilistic flip onset is varied over $\{1, 2, 3, 4\}$, and the logical row-space size is swept over $\{64, 128, 256, 512, 1024\}$. The policy parameters are fixed to: uniform TRR threshold $= 3$, security-aware TRR $(t_c, t_n) = (3, 10)$, uniform PARA probability $= 0.01$, and security-aware PARA $(p_c, p_n) = (0.18, 0.02)$. This preliminary sweep identifies the hard but still interpretable operating region used by the main timing-aware study.

C. Timing-Aware Parameter Study

Based on the fault-model sweep, we focus the timing-aware study on four representative fault settings. These settings span two probability-shape functions, two logical row-space sizes, and two probabilistic flip-onset points, providing a compact yet representative set of conditions for tuning the mitigation policies under the timing-aware surrogate.

TABLE I
FAULT-MODEL SWEEP USED TO IDENTIFY INFORMATIVE STUDY REGIONS.

Parameter	Values
Hammer Threshold	$\{1, 2, 3, 4\}$
Probabilistic Flip Onset	$\{1, 2, 3, 4\}$
Probability Function	Sigmoid
Logical Row-space Size	$\{64, 128, 256, 512, 1024\}$
Uniform TRR Threshold	3
Security-Aware TRR	$(t_c, t_n) = (3, 10)$
Uniform PARA Probability	0.01
Security-Aware PARA	$(p_c, p_n) = (0.18, 0.02)$

TABLE II
REPRESENTATIVE FAULT SETTINGS USED IN THE TIMING-AWARE PARAMETER STUDY.

Tag	Hammer	Onset	Function	Rows
S64-S1	1	1	Sigmoid	64
S128-S1	1	1	Sigmoid	128
L64-S1	1	1	Linear	64
S64-S2	1	2	Sigmoid	64

D. Interpretation

A security-aware policy should not be judged only by total bit flips. Since it intentionally reallocates protection toward critical rows, its primary objective is to reduce critical-region bit flips. The timing-aware row-level surrogate adds a second axis of evaluation: whether this protection gain is achieved at acceptable timing cost.

VI. Results

A. Role of the Fault-Model Sweep

Before comparing tuned security-aware policies, we first use the broader fault-model sweep to identify which parameter regions remain informative under the current maintenance surrogate. This distinction is important because large portions of the parameter space are already relatively benign, in which case policy comparisons become less revealing. The fault-model sweep therefore functions primarily as a study-selection tool: it highlights the operating regions in which both protection effectiveness and timing-related overhead remain visible enough to support meaningful comparison.

B. Fault-Model Parameter Selection

We use the fault-model sweep to identify which parameter regions remain informative under the updated row-level maintenance surrogate. Figures 3 and 4 show representative slices of this sweep. Figure 3 examines sensitivity to hammer threshold under a fixed hard setting with probabilistic flip onset 1 and logical row-space size 64, while Figure 4 examines sensitivity to logical row-space size under a fixed hard setting with hammer threshold 1 and probabilistic flip onset 1. Together, these figures compare protection effectiveness in terms of critical-region bit flips with timing overhead in terms of average read latency.

The dominant pattern is that the active region is concentrated in the smallest hammer thresholds and smallest logical

TABLE III
SWEPT TRR PARAMETERS IN THE TIMING-AWARE STUDY.

Parameter	Values
Uniform TRR threshold	3
Critical threshold	$\{2, 3, 4, 5\}$
Non-critical threshold	$\{2, 3, 4, 5, 6, 7, 8, 9, 10\}$

TABLE IV
SWEPT PARA PARAMETERS IN THE TIMING-AWARE STUDY. RANGES USE START:STEP:END NOTATION.

Parameter	Values
Uniform PARA probability	0.01
Critical probability	0.01:0.01:0.10, 0.12:0.02:0.20
Non-critical probability	0.01:0.01:0.10, 0.12:0.02:0.20

row spaces. When the hammer threshold is increased from 1 to 2, most configurations already collapse to a near-benign regime, and for thresholds 3 and 4 the curves are effectively flat. A similar effect appears in the row-space sweep: the strongest separation between policies occurs at 64 rows, a smaller but still visible difference remains at 128 rows, and by 256 rows and above the system has largely entered a mild regime in which all four policies behave similarly. Importantly, the timing metrics follow the same pattern as the security metrics: the hard fault settings are also the ones that activate noticeable latency and cycle overhead, making them the most informative settings for subsequent study.

This sweep is useful not because it introduces a new main result, but because it identifies which regions are worth studying in detail. For future experiments, the most informative settings are those with hammer threshold 1 or 2 and logical row-space size 64 or 128, with 256 still usable as a weaker secondary stress point. By contrast, thresholds 3 and 4 and large row spaces such as 512 or 1024 are generally too mild under the current model and therefore contribute little additional insight. Keeping the sigmoid probability function fixed is also justified by this result: once the highly active region is identified, the dominant source of variation is not the probability-shape function but the combination of hammer aggressiveness and interaction density.

Figure 5 isolates the effect of probabilistic flip onset. Compared with hammer threshold and logical row-space size, onset acts as a weaker but still meaningful stress knob. Moving the onset from 1 to 2 preserves noticeable separation in the hard 64-row setting, especially for PARA, while onsets 3 and 4 rapidly drive all policies toward a near-benign regime. This supports the use of onset values 1 and 2 in future studies and suggests that larger onset values are generally too mild to be informative under the current model.

C. Baseline Versus Best Security-Aware Configurations

We first compare each uniform baseline against the best security-aware configuration selected under the same rule: minimize critical-region bit flips first and, among ties, choose the configuration with the lowest average read latency. To

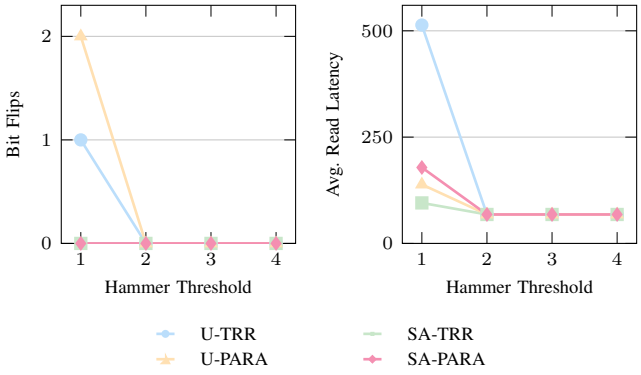

Fig. 3. Representative fault-model sweep slices under varying hammer threshold. The left panel shows critical-region bit flips, and the right panel shows average read latency.

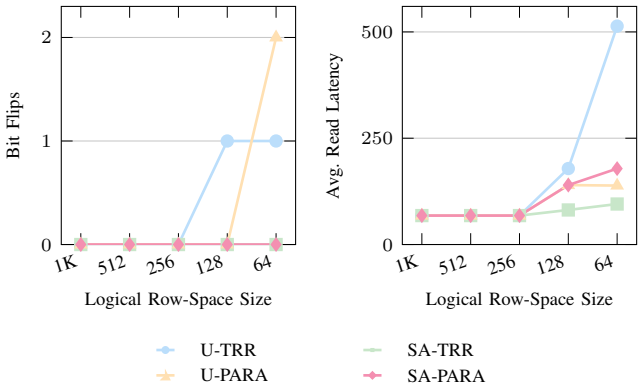

Fig. 4. Representative fault-model sweep slices under varying logical row-space size. The left panel shows critical-region bit flips, and the right panel shows average read latency.

make the comparison easier to read, Figures 6–9 organize the results by metric rather than by policy family. In each figure, the left panel shows TRR and the right panel shows PARA.

Figure 6 shows that both security-aware families can reduce critical-region bit flips, but they do so through different operating regimes. Security-aware TRR reaches zero critical flips in the three sigmoid-based settings and reduces the linear setting from 5 to 2 critical flips. Security-aware PARA also improves critical-region protection in the harder 64-row settings, reducing S64-S1 from 2 to 0, L64-S1 from 4 to 1, and S64-S2 from 1 to 0. In the milder S128-S1 case, uniform PARA already eliminates critical flips, so additional security-aware biasing is unnecessary.

Figure 7 separates the protection result from the latency cost. The difference between the two families becomes clearer in this organization. Security-aware TRR improves protection while also reducing average read latency in all four representative settings. This occurs because the best TRR region uses moderate critical thresholds and relaxed non-critical thresholds, avoiding excessive row-level maintenance on non-critical rows. Security-aware PARA, in contrast, usually pays additional latency for better protection because it increases the probability of maintenance on critical rows.

Figures 8 and 9 show the two timing-overhead proxies

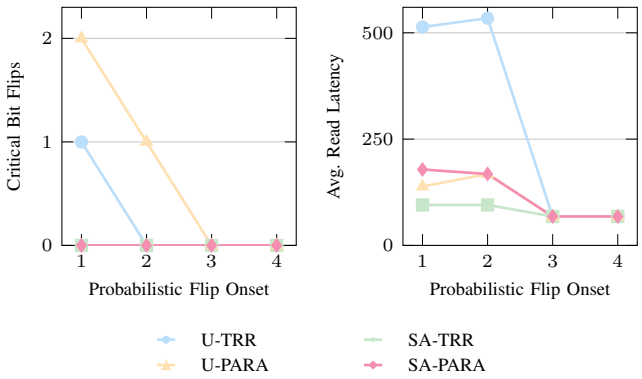

Fig. 5. Independent onset sweep slices under a fixed hard setting with hammer threshold 1 and logical row-space size 64. The left panel shows critical bit flips, and the right panel shows average read latency. Earlier flip-onset settings preserve noticeable protection and latency-overhead differences, while onsets 3 and 4 quickly collapse toward a mild regime.

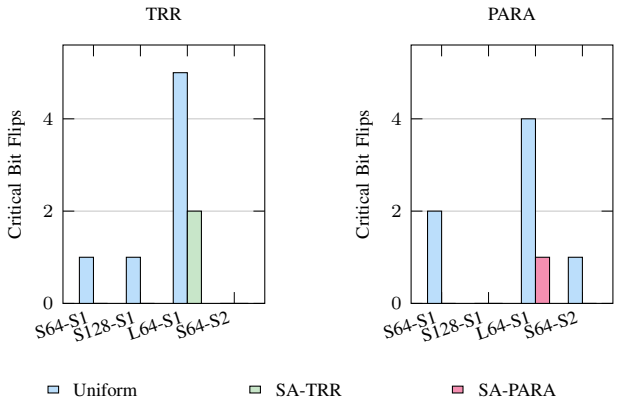

Fig. 6. Critical-region bit flips for the uniform baseline and the best security-aware configuration. The results are grouped by metric, with TRR on the left and PARA on the right.

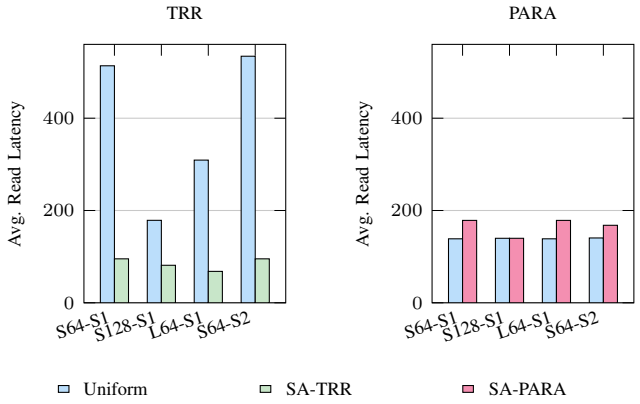

Fig. 7. Average read latency for the uniform baseline and the best security-aware configuration. TRR improves latency in the selected region, while PARA usually trades higher latency for stronger critical-row protection.

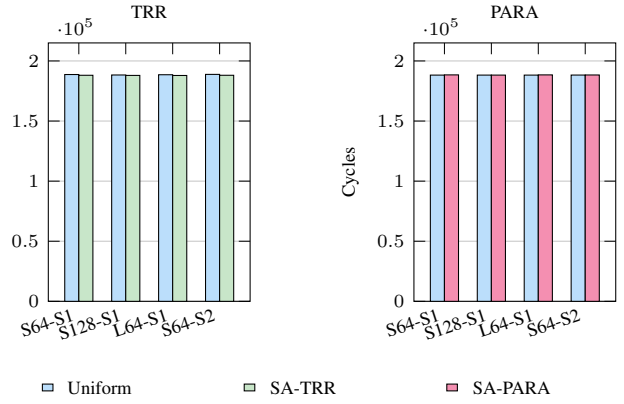

Fig. 8. Total memory-system cycles for the uniform baseline and the best security-aware configuration. The metric-level organization highlights that TRR reduces cycles, while PARA usually incurs a small cycle increase.

exposed by the maintenance surrogate. For TRR, security-aware selection consistently reduces total memory-system cycles and priority-queue occupancy relative to the uniform baseline. This confirms that its improvement is not limited to critical-row protection; it also suppresses unnecessary non-critical maintenance. For PARA, the best security-aware points usually increase both total cycles and priority-queue occupancy, especially in the harder 64-row settings, reinforcing the interpretation that PARA obtains protection mainly by increasing the frequency of probabilistic maintenance toward critical rows.

Overall, the metric-level organization shows that the two families represent different security-aware trade-off modes. TRR benefits from selective relaxation: it protects critical rows sufficiently while reducing unnecessary maintenance for non-critical rows. PARA benefits from selective intensification: it keeps the non-critical probability low while raising the critical probability, improving protection at the cost of additional maintenance pressure.

D. TRR Parameter Landscape

Figure 10 shows representative heatmaps for the hardest sigmoid 64-row setting. The left panel maps critical-region bit flips over the (t_c, t_n) grid, while the right panel maps average read latency over the same grid. The main pattern is clear. Making the critical threshold too small causes severe overhead, especially when the non-critical threshold is also small. By contrast, the best region appears around $t_c = 3$–4 combined with large t_n, which keeps critical protection strong while suppressing unnecessary maintenance to non-critical rows.

E. PARA Parameter Landscape

Figure 11 shows representative heatmaps for the S64-S1 setting. To keep the visualization readable, we plot a structured subset of the swept probability grid while preserving the key low-to-high transition region. The left panel shows critical-region bit flips, and the right panel shows average read latency. The main pattern is that good protection is achieved not by raising both probabilities together, but by keeping p_n low while increasing p_c. As p_n grows large, timing overhead

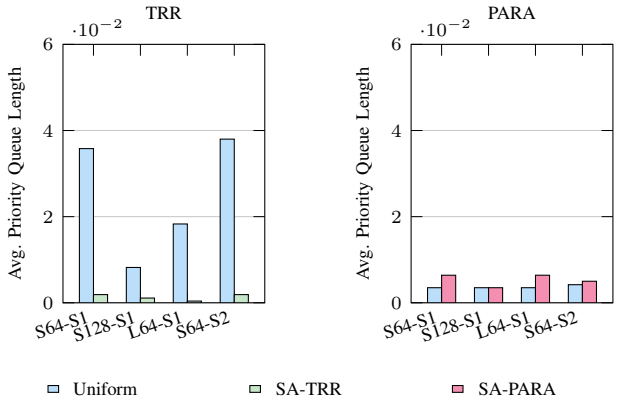

Fig. 9. Average priority-queue occupancy for the uniform baseline and the best security-aware configuration. Security-aware TRR greatly reduces maintenance pressure, whereas security-aware PARA increases it in the harder settings.

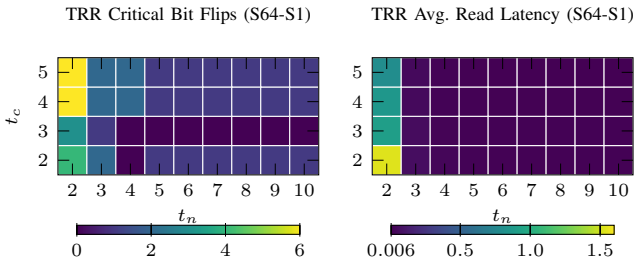

Fig. 10. Representative TRR heatmaps under the S64-S1 fault setting. The favorable region appears at moderate critical thresholds and relaxed non-critical thresholds.

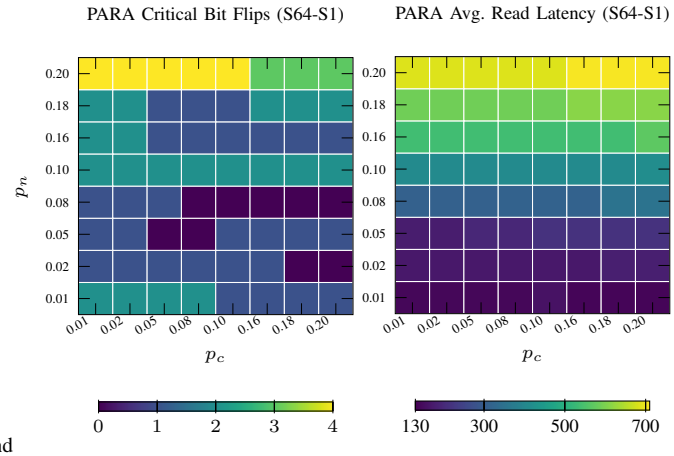

Fig. 11. Representative PARA heatmaps under the S64-S1 fault setting. Low non-critical probabilities and substantially higher critical probabilities define the most favorable trade-off region.

TABLE V
BEST-PERFORMING SECURITY-AWARE CONFIGURATIONS IN THE TIMING-AWARE STUDY.

Family	Setting	Parameters	Flips	Latency	Cycles
TRR	S64-S1	$t_c = 3,\ t_n = 10$	0	95.26	188082
TRR	S128-S1	$t_c = 3,\ t_n = 10$	0	81.29	187965
TRR	L64-S1	$t_c = 4,\ t_n = 10$	2	68.07	187910
TRR	S64-S2	$t_c = 3,\ t_n = 10$	0	95.26	188082
PARA	S64-S1	$p_c = 0.18,\ p_n = 0.02$	0	178.57	188403
PARA	S128-S1	$p_c = 0.01,\ p_n = 0.01$	0	139.78	188233
PARA	L64-S1	$p_c = 0.18,\ p_n = 0.02$	1	178.57	188403
PARA	S64-S2	$p_c = 0.16,\ p_n = 0.02$	0	167.94	188331

increases sharply and the quality of the protection region becomes less stable.

F. Parameter Recommendations

Table V summarizes the best-performing security-aware configurations at each representative fault setting. Two high-level patterns emerge.

For TRR, the best configurations are concentrated near $t_c = 3$ with a large non-critical threshold. This means that critical rows should be protected earlier than non-critical rows, but not so aggressively that row-level maintenance becomes excessive. For PARA, the best-performing points usually keep p_n small while using a substantially larger p_c. In other words, the important design choice is not to raise both probabilities, but to create a clear separation between them.

VII. Conclusion

In this project, we built a RowHammer evaluation framework on top of Ramulator2 that combines an activation-based logical fault model with a timing-aware row-level maintenance surrogate. This lets us evaluate not only whether a policy reduces critical-region bit flips, but also how that protection strength affects timing-related overhead such as read latency, total cycles, and priority-queue occupancy.

The timing-aware parameter study reveals different design patterns for the two mitigation families. Security-aware TRR performs best when critical rows are protected with a moderate threshold and non-critical rows are protected less aggressively; this configuration can reduce critical-region bit flips while substantially lowering timing overhead relative to uniform TRR. Security-aware PARA, in contrast, shows a clearer protection-versus-performance trade-off: the best configurations keep the non-critical protection probability low while assigning a substantially higher probability to critical rows. These results suggest that security-aware mitigation is most effective when it explicitly reallocates limited protection effort toward critical rows rather than applying equally aggressive protection everywhere. The fault-model sweep further shows that the most informative operating region lies in the hard regime with small hammer thresholds and small logical row-space sizes, since much of the broader parameter space is already too mild to produce meaningful separation between policies.

References

[1] Y. Kim *et al.*, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," in *Proc. ISCA*, 2014.
[2] H. Luo *et al.*, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *arXiv preprint arXiv:2308.11030*, 2023.